C++ SIMD intrinsics cheat sheet

x86 / GCC & Clang | Include <immintrin.h> | Project 1 desk reference

An intrinsic is a compiler-provided function exposing CPU operations. A lane is one element inside a SIMD value. Operations below act independently on matching lanes; add is not a sum across lanes. Local CPU: Intel i7-13800H; lscpu reports SSE/SSE2, AVX, AVX2 and FMA, but no AVX-512. [1-6]

Types and naming

Type	Width	Elements / lanes	Minimum ISA for arithmetic shown
__m128	128 bits	4 x float	SSE
__m128d	128 bits	2 x double	SSE2
__m256	256 bits	8 x float	AVX
__m256d	256 bits	4 x double	AVX
__m256i	256 bits	e.g. 8 x int32	AVX2 for add_epi32 / mullo_epi32

_mm_ = 128-bit family here; _mm256_ = 256-bit. ps = packed float; pd = packed double; ss/sd = scalar float/double forms, not full-lane arithmetic. [2-4]

128-bit lookup: SSE float / SSE2 double

Task	float form	double form
All lanes zero	`_mm_setzero_ps()`	`_mm_setzero_pd()`
Broadcast one scalar	`_mm_set1_ps(s)`	`_mm_set1_pd(s)`
Load without vector alignment	`_mm_loadu_ps(p)`	`_mm_loadu_pd(p)`
Store without vector alignment	`_mm_storeu_ps(p, v)`	`_mm_storeu_pd(p, v)`
Load, 16-byte aligned pointer	`_mm_load_ps(p)`	`_mm_load_pd(p)`
Store, 16-byte aligned pointer	`_mm_store_ps(p, v)`	`_mm_store_pd(p, v)`
Lane-wise add	`_mm_add_ps(a, b)`	`_mm_add_pd(a, b)`
Lane-wise multiply	`_mm_mul_ps(a, b)`	`_mm_mul_pd(a, b)`

256-bit lookup: AVX float / double

Task	float form	double form
All lanes zero	`_mm256_setzero_ps()`	`_mm256_setzero_pd()`
Broadcast one scalar	`_mm256_set1_ps(s)`	`_mm256_set1_pd(s)`
Load without vector alignment	`_mm256_loadu_ps(p)`	`_mm256_loadu_pd(p)`
Store without vector alignment	`_mm256_storeu_ps(p, v)`	`_mm256_storeu_pd(p, v)`
Load, 32-byte aligned pointer	`_mm256_load_ps(p)`	`_mm256_load_pd(p)`
Store, 32-byte aligned pointer	`_mm256_store_ps(p, v)`	`_mm256_store_pd(p, v)`
Lane-wise add	`_mm256_add_ps(a, b)`	`_mm256_add_pd(a, b)`
Lane-wise multiply	`_mm256_mul_ps(a, b)`	`_mm256_mul_pd(a, b)`

Signature key: p is const float*/const double* for loads, float*/double* for stores. s is a scalar float/double. a, b, v are matching SIMD types above. Loads, zero/broadcast, add and multiply return that SIMD type; stores return void. Example: __m256 _mm256_loadu_ps(const float* p). [2-4]

Safety, examples & build flags

FMA is a separate feature [5]

Intrinsic call	Result type	Meaning / requirement
_mm256_fmadd_ps(a, b, c)	__m256	Lane-wise a*b+c; AVX + FMA
_mm256_fmadd_pd(a, b, c)	__m256d	Lane-wise a*b+c; AVX + FMA
_mm_fmadd_ps(a, b, c)	__m128	Same operation, 4 floats; FMA
_mm_fmadd_pd(a, b, c)	__m128d	Same operation, 2 doubles; FMA

FMA rounds once, unlike separate multiply then add; low bits may differ. The Makefile's -ffp-contract=off stops automatic contraction, not explicit FMA intrinsics. Compare finite outputs using justified absolute + relative tolerances; reject unexpected NaNs/infinities. AVX2 does not itself imply FMA support. [5, 7]

Tiny example: scale exactly one block, not a kernel

Assume src and dst each provide at least 8 live float elements; no special vector alignment is needed. This is illustrative code inside a function, with no loop or assignment implementation:

```
#include <immintrin.h>  // place at file scope
// Inside your function, with valid src and dst pointers:
__m256 v = _mm256_loadu_ps(src);
__m256 two = _mm256_set1_ps(2.0f);
_mm256_storeu_ps(dst, _mm256_mul_ps(v, two));
```

Alignment and bounds: loadu/storeu remove the 16/32-byte alignment requirement, not the need for valid elements. Every full load/store touches 4/8 floats or 2/4 doubles. Never read past n, even if unwanted lanes are ignored. [2-4]

Tails: process only complete blocks, then handle remaining elements with scalar operations. For 19 floats and width 8, there are 2 full blocks and 3 scalar elements. Start the scalar remainder at element 16. Do not pad the length upward without allocating and defining the extra elements.

Aligned storage example: alignas(32) float block[8]; initializes no values by itself. Initialize before reading. Ordinary std::vector<float> does not promise 32-byte alignment; even an aligned base loses that alignment at base+1. Use unaligned forms first. Keep input/output ranges separate unless your algorithm deliberately supports overlap.

Compile with the existing Makefile [7, 8]

```
make -B CXX=g++ MODE=native
# Explicit AVX-only target for the examples (separate output):
make -B CXX=g++ MODE=avx BUILD_DIR=build/avx CXXFLAGS='-mavx'
# For FMA: use CXXFLAGS='-mavx -mfma'; for AVX2 integers: '-mavx2'
```

Replace g++ with clang++ if desired. MODE=avx adds -mavx; MODE=native adds -march=native to both sides. Scalar .cpp files disable auto-vectorization; *_simd.cpp files contain your intrinsics. No-vectorization flags do NOT disable intrinsics: keep scalar files ordinary C++. Both variants share -O3 and the ISA. AVX floats need AVX, not AVX2. Integer examples: _mm256_add_epi32(a,b) and _mm256_mullo_epi32(a,b) require AVX2. [6]

Target flags enable compilation; they add no runtime dispatch. A native/AVX binary can fail on another CPU or unsupported OS setup. Use a compatible target or a separately dispatched fallback when sharing. Inspect saxpy_simd.o with objdump -d -C; vector instructions do not prove speedup. [7, 8]

Auto-vectorized comparison: still required separately by Project 1. Later add ordinary-loop *_auto variants without scalar disabling flags; explicit SIMD does not replace them.

Primary references (clickable)

[1] Intel Intrinsics Guide: https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html
[2] SSE: https://clang.llvm.org/doxygen/xmmintrin_8h.html
[3] SSE2: https://clang.llvm.org/doxygen/emmintrin_8h.html
[4] AVX: https://clang.llvm.org/doxygen/avxintrin_8h.html
[5] FMA: https://clang.llvm.org/doxygen/fmaintrin_8h.html
[6] AVX2: https://clang.llvm.org/doxygen/avx2intrin_8h.html
[7] GCC x86 flags: https://gcc.gnu.org/onlinedocs/gcc-13.4.0/gcc/x86-Options.html
[8] Clang manual: https://clang.llvm.org/docs/UsersManual.html